Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Justin Bennighoff	justincb@udel.edu
Friend (optional)	Nate	aspenchk@udel.edu

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea

and a paragraph describing how that paper supports your design concept.
        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider the project below.
        - Create an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the chempy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.

Civil engineering, create something that deals with traffic in different ways such as highways, side roads and bridges

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There is a traffic flow simulator (link) that has different interchanges and simulates different traffic flows. Cities skylines is a game that you build a city and that also has city planning and traffic management.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

How different types of roads impact traffic flow or different types of interchanges.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

Taking in a number for cars per hour and what type of road/interchange then returning a simulation based on those values.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The number of cars per hour, type of interchange, type of road.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

It will take in that data and produce a simulation that shows traffic flow with random movements.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

A visual of the flow of traffic for the road and interchange they choose.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Cars_per_hour, road_type, interchange_type,

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Details I need - how much traffic, which intersection, which road type. *Details I will ignore - type of vehicle, color of things, density of area.*

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The visual aspect and how I will be able to show the traffic model.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will make sure there are doc strings and other documentation for every aspect so I know how everything works within the code.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

Build something that I can show off to people in my major and on my linked in.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a simple web tool that helps civil engineers see traffic flow. What are some common problems that civil engineers face when trying to solve traffic flow and what are 3 simple features a basic tool should have.

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

Have a traffic flow map, data dashboard, and traffic light simulator.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

How will I be able to do the complex visuals just in python and drafter

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I think it has become more focused and might be dumbed down a little bit if I can't figure the visual aspect out.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will use my LLM, my friends, and the discord.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

Ask the LLM more questions and tell it how the code isn't working and see what it says. Ask my friends if they can see what's going wrong and if they can't, ask the discord about it.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

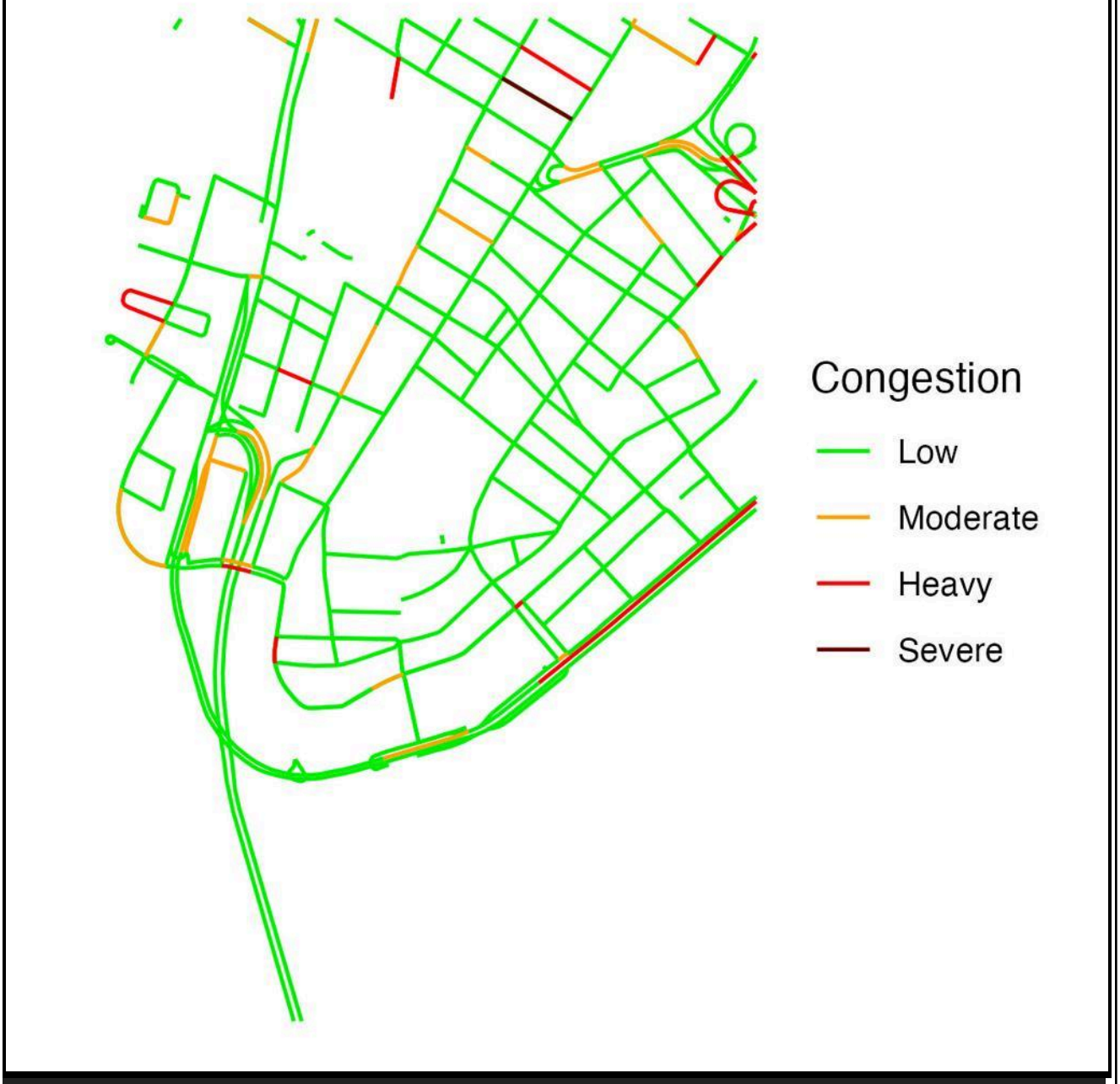
I want to build a simple web tool that helps civil engineers see traffic flow. What are some common problems that civil engineers face when trying to solve traffic flow and what are 3 simple features a basic tool should have.

A simple traffic flow web tool could help civil engineers understand where traffic problems happen, why they happen, and how to improve road conditions.

Common problems civil engineers face

1. Traffic congestion: Too many vehicles using the same road can cause traffic jams, especially during rush hour.
2. Poor traffic light timing: Traffic signals that aren't timed efficiently can create long lines of waiting vehicles.
3. Accidents and road closures: Crashes, construction, and blocked lanes can interrupt traffic flow and cause delays.
4. Difficulty analyzing traffic data: Engineers need to understand vehicle counts, speeds, and traffic patterns, but collecting and interpreting this information can be challenging.
5. Changing traffic patterns: Traffic volume changes depending on the time of day, weather, and special events.

Three simple features your web tool should have

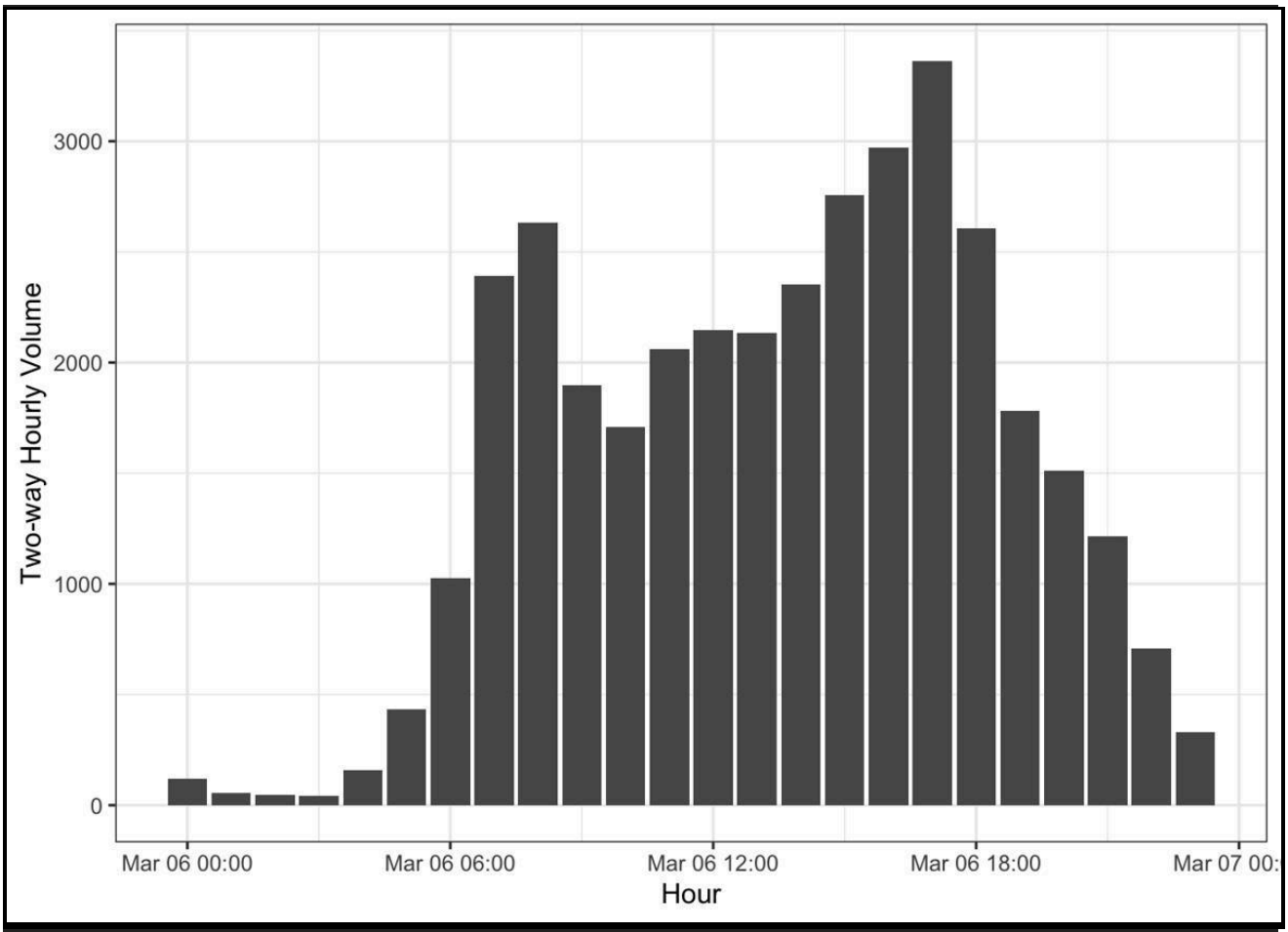


1. Traffic Flow Map

Display roads using colors to represent traffic conditions.

- Green: Light traffic
- Yellow: Moderate traffic
- Red: Heavy traffic

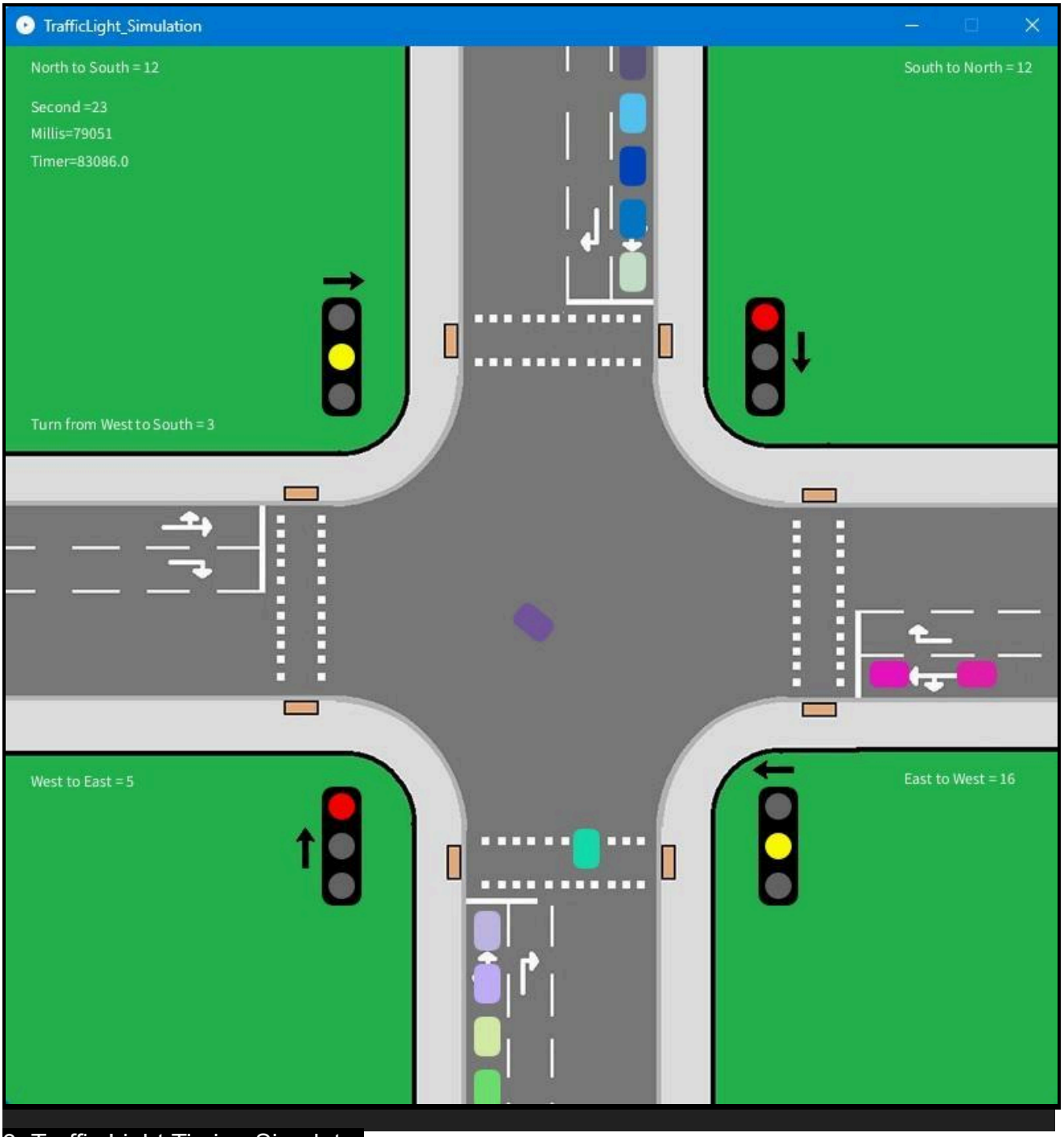
This helps engineers quickly identify congested areas.



2. Traffic Data Dashboard

Show basic information such as vehicle counts, average speeds, and busiest hours using simple graphs.

This helps engineers identify when traffic is heaviest and compare different roads.



3. Traffic Light Timing Simulator

Allow users to adjust how long traffic lights stay green or red and compare estimated vehicle delays.

This helps engineers explore whether changing signal timing could improve traffic flow.

How this connects to programming

Your *Python Bakery* textbook explains the Input → Processing → Output model. You can use that same idea to plan your traffic tool.

Stage	Traffic tool example
Input	Number of cars, average speed, traffic light timing
Processing	Calculate traffic averages or compare traffic conditions
Output	Display graphs, road colors, or estimated delays

For a beginner project, you could start with manually entered or sample traffic data instead of live traffic sensors.

A question to help plan your tool

Which problem do you most want your tool to solve?

Finding roads with heavy traffic

Improving traffic light timing

Understanding traffic patterns throughout the day

i kind of want to understand how different interchanges with different road types flow with visuals

That sounds like a more interesting direction! Instead of only showing traffic congestion, your tool could become an interactive road interchange visualizer where civil engineers can see how cars move through different road designs.

The main question your tool could explore is: How does the design of an interchange affect traffic flow?

1. Different interchange designs to visualize



Diamond interchange

Connects a highway to a smaller road using four ramps. Traffic lights or stop signs often control where ramps meet the smaller road.

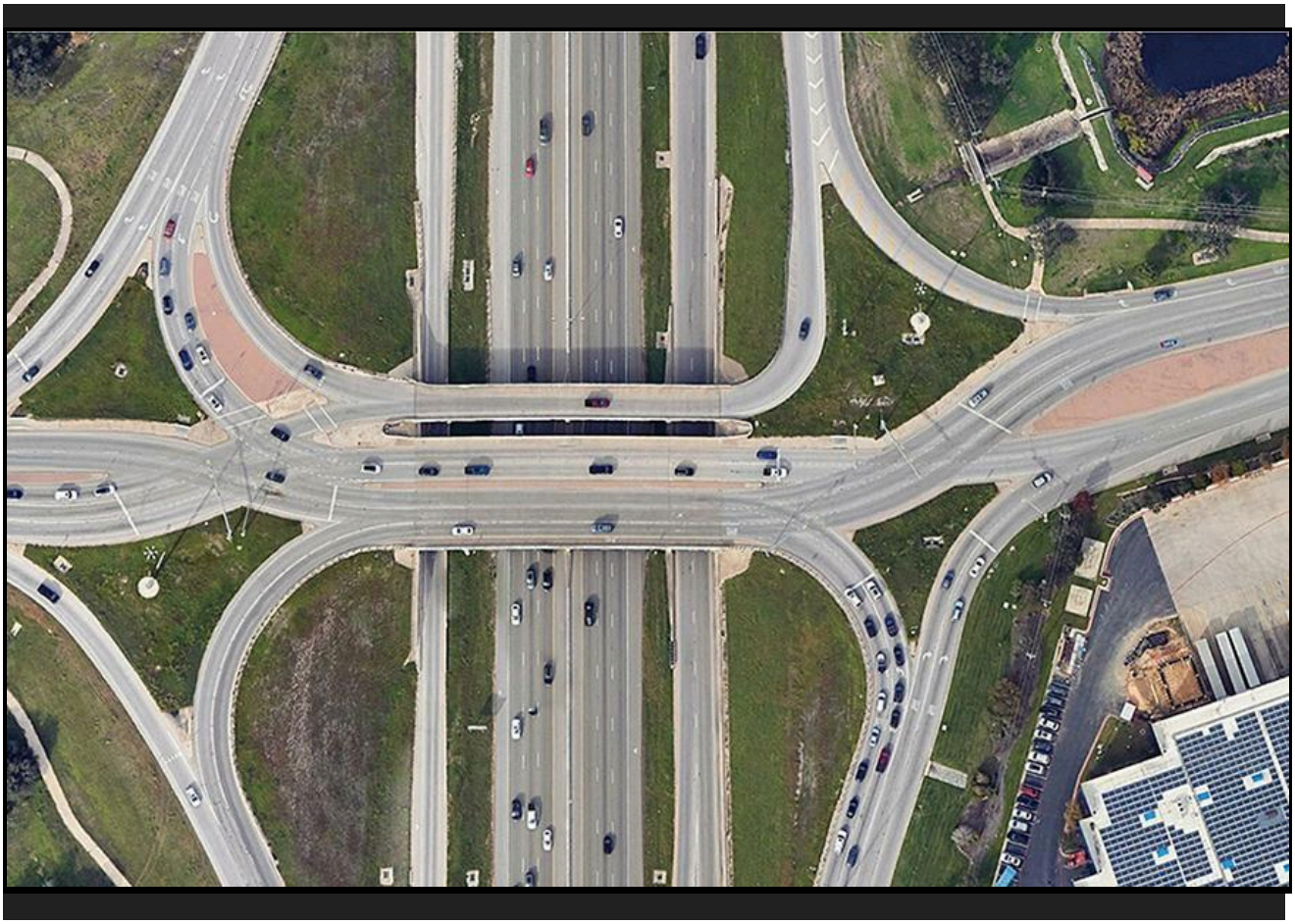
Interesting to visualize: Cars waiting at ramp intersections.



Cloverleaf interchange

Uses looping ramps to connect two highways without traditional traffic lights.

Interesting to visualize: Cars merging and weaving between entrance and exit ramps.



Diverging diamond interchange

Temporarily moves traffic to the opposite side of the roadway at a highway crossing to simplify certain turning movements.

Interesting to visualize: How traffic moves through the crossover signals.

2. Three features I'd consider

Interactive interchange selection: Choose between diamond, cloverleaf, and diverging diamond designs, with a simplified overhead diagram of each.

Animated traffic movement: Show small colored vehicles moving along roads, merging, turning, and sometimes waiting. You could adjust the number of vehicles entering the

interchange.

Traffic flow comparison: Display estimated throughput, queue lengths, and average delay to compare different interchange designs under the same traffic conditions.

3. What the tool might look like

Interchange Traffic Explorer

Concept preview

Interchange type

Diamond

Cloverleaf

Diverging



Illustrative road layouts, not engineering-scale designs or a traffic simulation.

Traffic volume

55%

Moderate demand

Illustrative vehicle density only

This connects to your *Python Bakery* textbook's idea of abstraction: you don't need to model every detail of a real interchange. You can start by representing roads, vehicles, and movements using simplified data and visuals.

One important design question

Would you rather build a tool where:

A. Cars move through different interchanges, and you watch how traffic behaves.

B. You design your own interchange by adding roads, ramps, and lanes.

C. You compare two interchange designs side by side to see which handles traffic better.

how will I be able to do the visuals of this just in python using drafter

Worked for 10s

You can definitely build a visual traffic interchange tool using Python and Drafter, without having to learn JavaScript or build a complicated website!

The important thing is understanding what Drafter can display and how you can use Python to control those visuals.

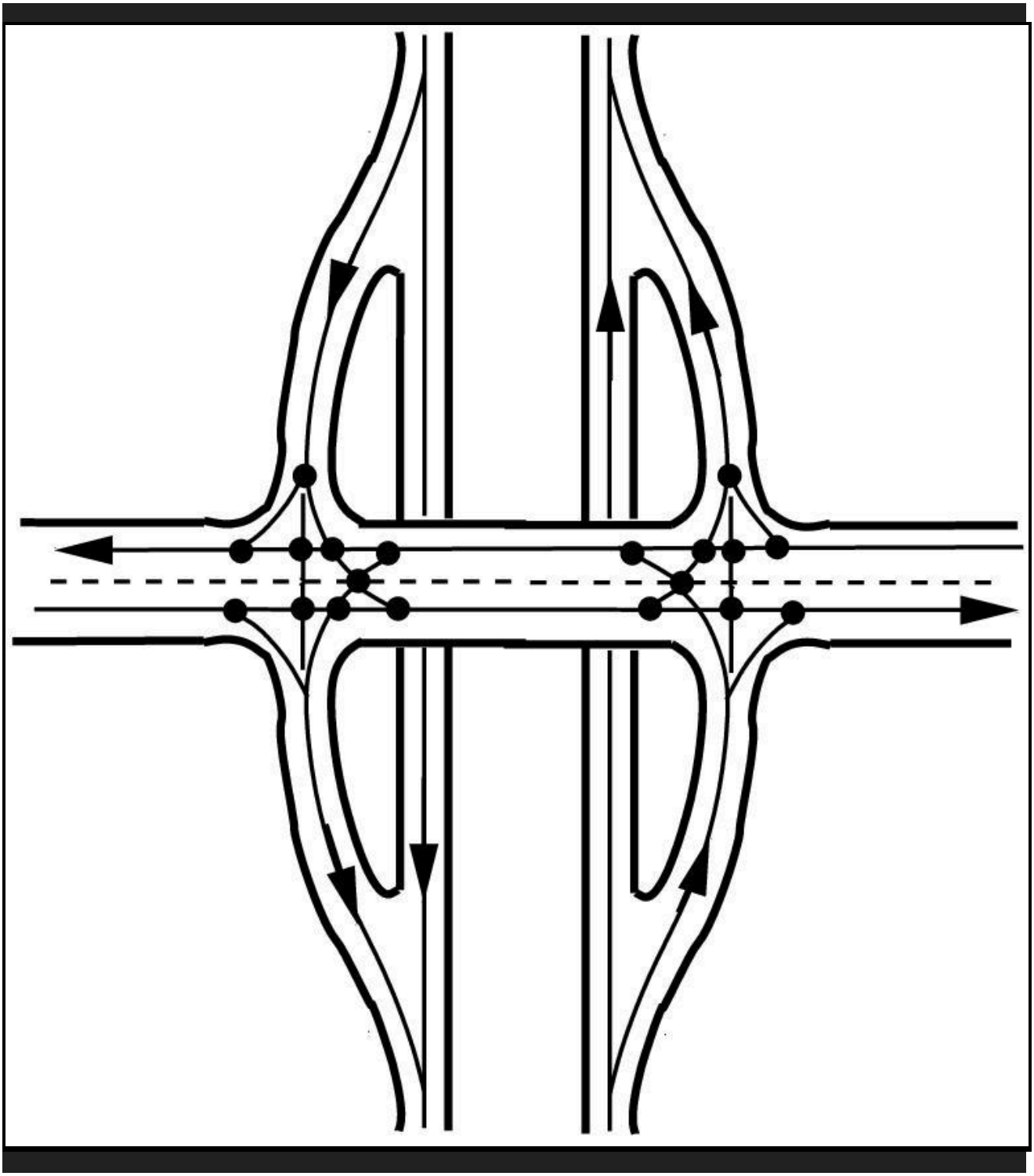
Before getting into the technical details, think about this:

Do you need cars to move continuously across the screen, or would showing their positions changing each time the user clicks a button be enough?

That distinction matters because Drafter is primarily designed for interactive web pages, not complex real-time animations.

1. How you could create visuals in Drafter

There are three beginner-friendly approaches.

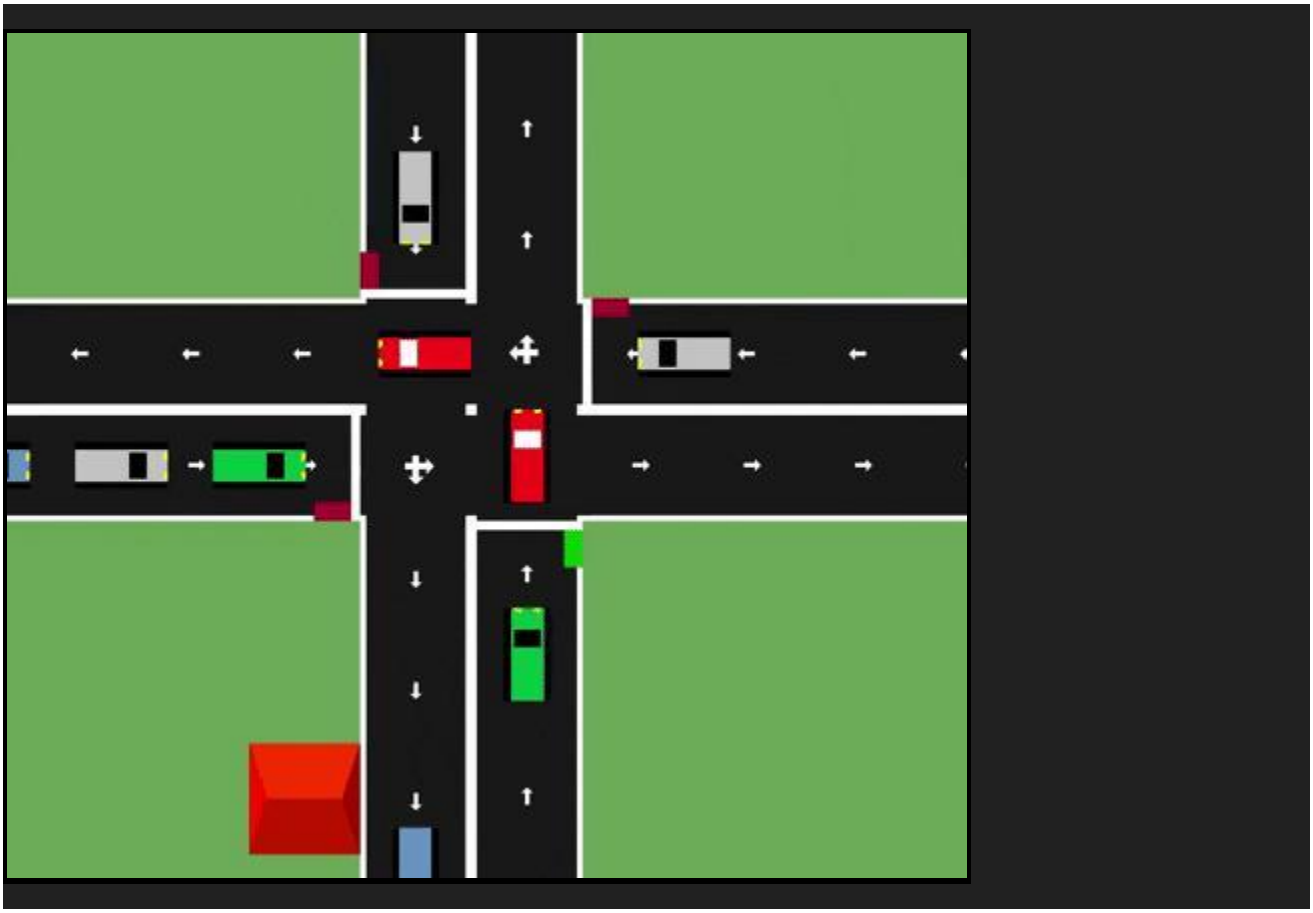


Option 1: Road images

Use an image of each interchange as the background.

Drafter can display an image on a webpage, and Python can control which image appears based on the user's selection.

This is probably the easiest starting point.

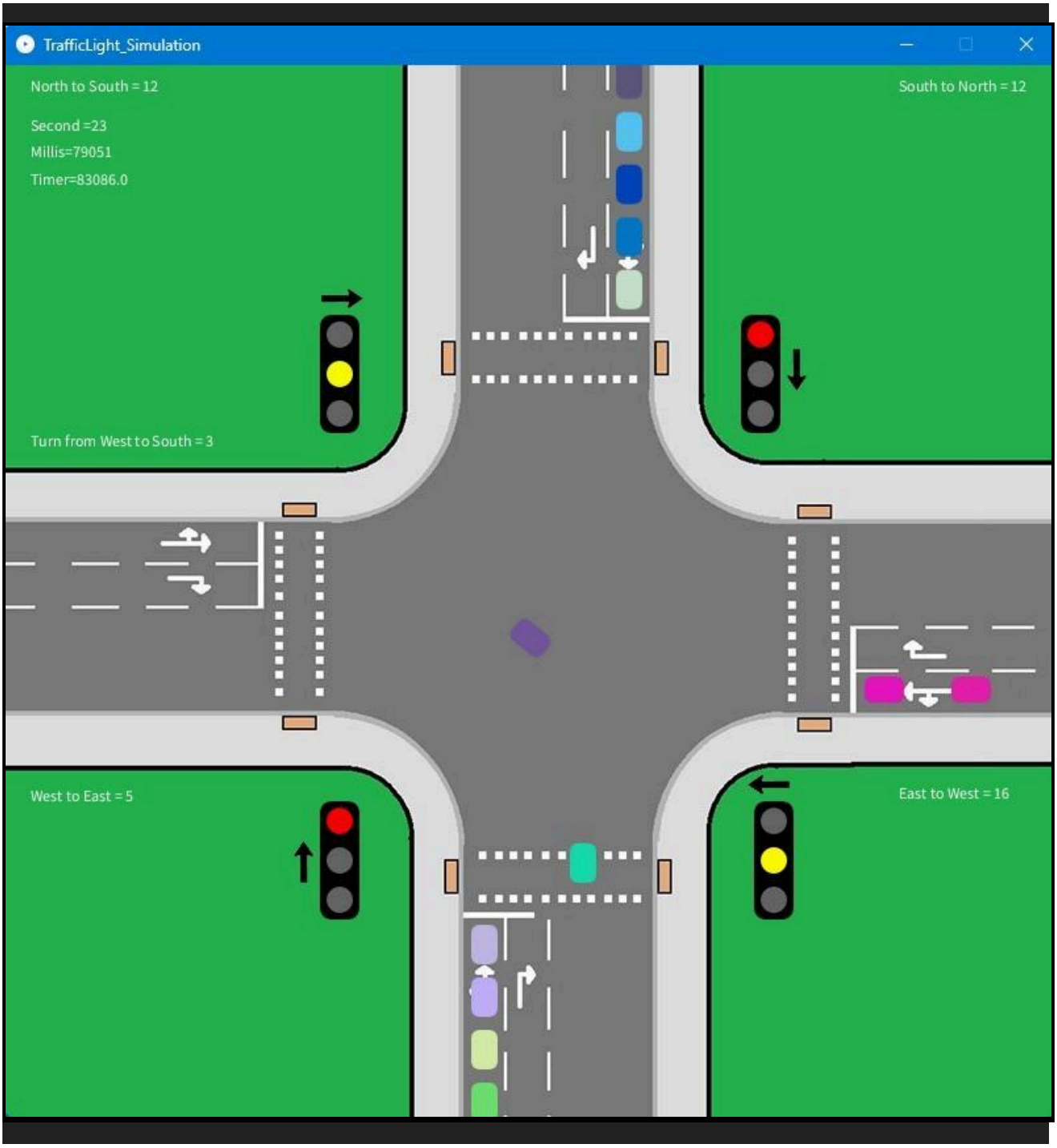


Option 2: Python-generated diagrams

Use Python to create simplified roads and colored dots or rectangles representing vehicles.

You could use a Python drawing library to generate images, then display those images through Drafter.

This gives you more control over vehicle positions.



Option 3: Interactive traffic diagrams

Display a diagram that changes when the user clicks a button.

For example, a user could increase traffic volume, change the interchange type, or advance

the simulation by one step.

Python would update the underlying traffic data and Drafter would display the new result.

2. How Python and Drafter would work together

Your textbook's Drafter section explains that a `Button` can call a `@route` function, update the program's state, and display a new `Page`.

bakery_complete.md

For your interchange tool, that means:

User selects an interchange

For example, a cloverleaf

Drafter calls a Python route

The route reads or changes the state

Python selects or generates the visual

A road diagram with vehicle positions

Drafter displays the updated page

The user sees the new traffic conditions

3. What about actual moving cars?

This is where the limitations matter.

With basic Drafter, I would start with a step-by-step simulation rather than continuous animation.

For example, imagine a car approaching an interchange. Every time the user presses "Next

Step," the car moves farther along its route.

Vehicle movement example

Step 1 of 5



This is an illustration of the interaction you could aim to reproduce in Drafter, not Drafter code.

For smoother animation, you would need to investigate whether your course environment permits additional rendering or browser-side tools. The textbook establishes Drafter's page updates, but not a built-in real-time animation engine.

4. What I would build first

Start with one interchange, a static diagram, and a button that changes the displayed vehicle positions. Once that works, you can consider additional road types and traffic conditions.

One question will help determine the right visualization approach:

Are you allowed to use other Python libraries, such as Pillow or Matplotlib, alongside Drafter, or does your project require Drafter alone?

That determines whether you can generate detailed road drawings in Python or should rely mainly on images and Drafter's built-in components.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.